

PRODUCTION MEDICAL DOCUMENT RAG

MediQuery RAG

Interview Explanation Notes

Hinglish guide: project pitch, architecture, LangChain pipeline, FAISS + SQLite mapping, medical safety, trade-offs, scaling, and interview Q&A.;

FASTAPI	LANGCHAIN	SQLITE	FAISS
---------	-----------	--------	-------

Prepared for: Ashish | Project: MediQuery RAG

INTERVIEW NOTES

1. Sabse pehle: project ko kaise introduce karna hai

Interviewer ko implementation details se pehle problem, solution, trust, aur production thinking samjhao.

30-second version

Say this

MediQuery RAG ek medical document question-answering system hai. User PDF upload karta hai, backend document ko page-wise parse aur chunk karta hai, LangChain embeddings ke through FAISS mein index karta hai, aur selected documents se grounded answers generate karta hai. Har answer ke saath document name, page number, chunk preview aur relevance score citation ke roop mein return hota hai.

2-minute version

Maine MediQuery RAG ko ek production-style full-stack project ki tarah design kiya. Frontend React, TypeScript, Tailwind, TanStack Query aur Axios par hai. Backend FastAPI, SQLAlchemy aur SQLite use karta hai. Upload ke baad PDF pehle validate aur local storage mein safely save hoti hai. LangChain ka PyMuPDFLoader page-wise text load karta hai, RecursiveCharacterTextSplitter overlapping chunks banata hai, aur OpenAI-compatible OpenAIEmbeddings un chunks ko vectors mein convert karta hai.

LangChain FAISS vector store semantic search ke liye use hota hai. SQLite business metadata ka source of truth hai, jabki FAISS semantic vectors rakhta hai. Chat request mein selected document IDs ke basis par retrieval filter hota hai. Top relevant chunks ChatPromptTemplate ke strict medical rules ke saath ChatOpenAI ko diye jaate hain. Final citation model se invent nahi hoti - woh retrieved SQLite chunk metadata se deterministically attach hoti hai. Isse answer explainable aur auditable banta hai.

Medical safety ke liye system outside knowledge forbid karta hai, insufficient context mein fixed fallback deta hai, risky diagnosis/dosage/treatment queries detect karta hai, aur professional medical advice warning enforce karta hai.

Problem Large PDFs manually search karna slow hai, aur generic LLM hallucinate kar sakta hai.	Solution Document-grounded semantic retrieval plus citation-backed generation.
Trust Every claim ko filename, page, chunk, preview aur score se trace kar sakte hain.	Production angle Thin routes, services, env config, error handling, tests, safe storage, persistent index.

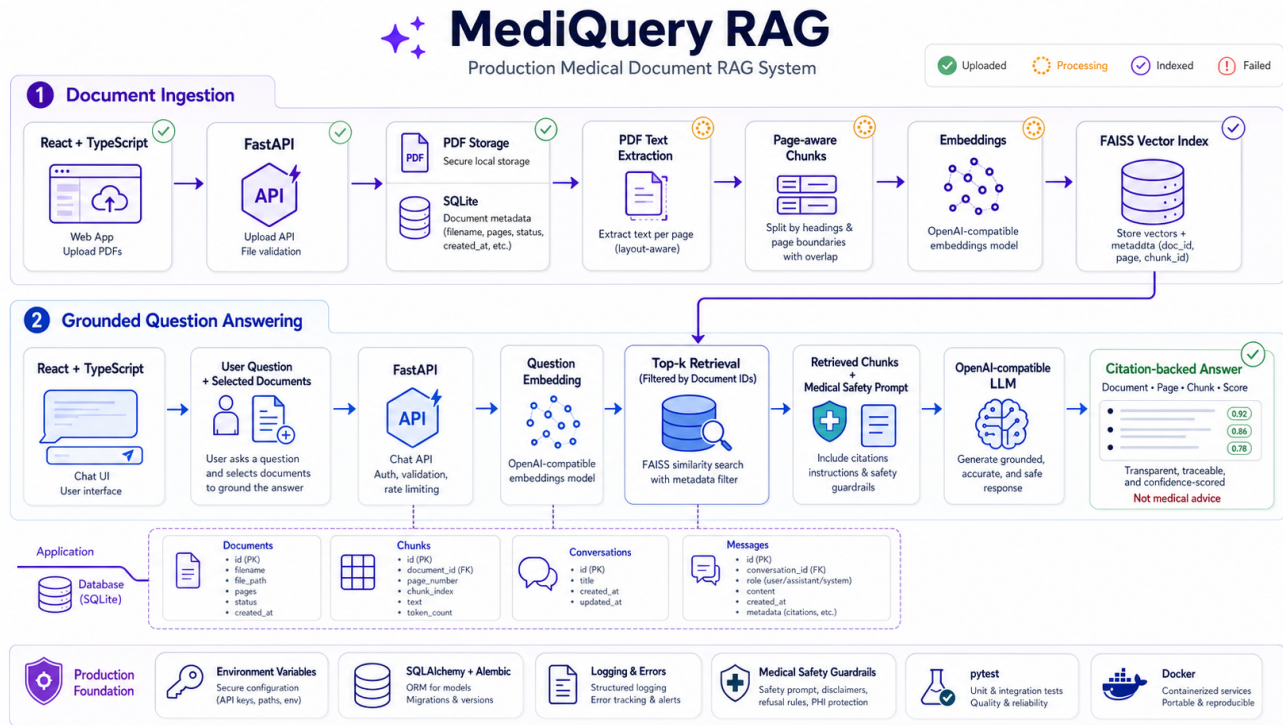
Golden line

This is not just an LLM wrapper. It is a retrieval, provenance, safety, and API orchestration system.

INTERVIEW NOTES

2. High-level architecture

Pehle overall system boundary samjhao, phir ingestion aur chat ko do separate pipelines ki tarah explain karo.



- Frontend sirf backend API se baat karta hai. API keys browser bundle mein kabhi nahi jaati.
- FastAPI HTTP validation, dependency injection, CORS aur service orchestration handle karta hai.
- SQLite documents, chunks, conversations aur messages ka durable source of truth hai.
- LangChain RAG components loading, splitting, embeddings, FAISS retrieval, prompt aur chat model handle karte hain.
- Local PDF storage original source preserve karta hai; FAISS semantic vectors persist karta hai.

Interview structure

Architecture explain karte waqt bolo: system ke do main flows hain - offline ingestion flow aur online question-answering flow.

INTERVIEW NOTES

3. PDF upload se indexed document tak

Ingestion comparatively expensive hai; isliye har stage ka clear responsibility aur failure state hai.



Step-by-step

- Upload endpoint multipart PDF receive karta hai. Route thin hai; actual logic DocumentService mein hai.
- FileStorage extension, MIME type, %PDF- magic bytes aur 50 MB streaming limit validate karta hai.
- UUID-prefixed stored filename collision aur path traversal risk reduce karta hai.
- Document row pehle processing status ke saath commit hoti hai, taaki failure UI mein visible rahe.
- LangChainPDFLoader, PyMuPDFLoader ko page mode mein run karta hai. Har page ek LangChain Document banta hai.
- LangChainChunker page Documents ko RecursiveCharacterTextSplitter se split karta hai.
- Chunk rows SQLite mein flush hote hain; flush se IDs multi hain bina transaction complete kiye.
- SQLite chunk_id LangChain Document metadata mein add hota hai, phir OpenAIEmbeddings aur FAISS indexing hoti hai.
- FAISS position vector_id column mein save hota hai; successful commit ke baad status indexed hota hai.
- Expected failure par transaction rollback hota hai aur document failed status plus safe error message paata hai.

Why processing status first?

Agar parsing ya embedding provider fail ho jaye, user ko invisible failure nahi milta. Document library mein failed state aur diagnostic message show hota hai.

```
POST /api/documents/upload
PDF -> validation -> local file -> processing row
-> PyMuPDFLoader -> RecursiveCharacterTextSplitter
-> SQLite chunks -> OpenAIEmbeddings -> LangChain FAISS
-> indexed status
```

INTERVIEW NOTES

4. LangChain exactly kahan use hua hai

Framework ka naam bolna enough nahi hai. Component, responsibility aur reason clearly batao.

PyMuPDFLoader Saved PDF ko standard LangChain Documents mein load karta hai. Page metadata citation provenance preserve karta hai.	RecursiveCharacterTextSplitter Paragraph -> line -> sentence -> word separators recursively try karke bounded chunks banata hai.
OpenAIEmbeddings Chunk text aur user query ko same vector space mein map karta hai. base_url se compatible providers support hote hain.	LangChain FAISS Document text plus metadata ko vector index ke saath manage karta hai aur scored metadata-filtered search deta hai.
ChatPromptTemplate System medical rules, retrieved context, question aur safety instruction ko repeatable prompt format mein rakhta hai.	ChatOpenAI OpenAI-compatible Chat Completions model invoke karta hai; model, key aur base URL environment-driven hain.

Why not one opaque RetrievalQA chain?

MediQuery retrieval ko explicit rakhta hai: pehle selected-document filtering, phir SQLite validation, phir prompt invocation, phir deterministic citations. Interview mein bolo ki healthcare context mein observability aur provenance convenience se zyada important hai. Opaque chain mein debugging aur exact citation control mushkil ho sakta hai.

Strong answer

LangChain integration abstraction ke liye use kiya, lekin business rules, safety, transactions aur citation ownership application ke paas rakha.

OpenAI-compatible configuration

```
ChatOpenAI(model=CHAT_MODEL, api_key=OPENAI_API_KEY,
base_url=OPENAI_BASE_URL, temperature=0)
OpenAIEmbeddings(model=EMBEDDING_MODEL,
api_key=OPENAI_API_KEY, base_url=OPENAI_BASE_URL)
```

INTERVIEW NOTES

5. Chunking aur embeddings ko kaise explain karein

Yeh do concepts almost har RAG interview mein pooche jaate hain.

Chunking

LLM ko poori PDF dena inefficient aur unreliable ho sakta hai. Isliye text ko retrieval units mein split kiya jaata hai. Project around 900 characters ka chunk aur 180 characters ka overlap use karta hai. Split page boundary cross nahi karta, isliye page citation accurate rehti hai.

Chunk too small Context fragmented hoga, retrieval noisy ho sakta hai, aur answer ke liye incomplete sentence mil sakta hai.	Chunk too large Embedding broad ho jaata hai, irrelevant text retrieve ho sakta hai, token cost badhti hai.
Why overlap Boundary ke paas sentence ya concept cut hone par adjacent chunk mein context repeat hota hai.	Why recursive Natural separators ko priority milti hai; fixed character slicing ke comparison mein semantic coherence better hota hai.

Embeddings

Embedding ek dense numerical vector hai jo text ka semantic meaning represent karta hai. Similar meaning wale texts vector space mein paas hote hain. Query aur document chunks same embedding model se encode hone chahiye, warna similarity meaningful nahi hogi.

Simple analogy

Embeddings ko semantic coordinates samjho. FAISS question ke coordinates ke nearest chunk coordinates find karta hai.

- Embedding model environment variable se configurable hai.
- OpenAI-compatible base URL provider switch ko code change ke bina possible banata hai.
- Embedding model change hone par existing index rebuild karna zaroori hai because vector dimensions/space change ho sakte hain.
- Provider errors 502/503 style application errors mein convert hote hain; secrets logs mein expose nahi hote.

INTERVIEW NOTES

6. SQLite aur LangChain FAISS ka connection

Yeh project ka most important data-design answer hai.

One-line answer SQLite is the source of truth for business metadata; FAISS is the optimized semantic search index.

SQLite stores Document status, filenames, file path, chunk text, page number, chunk index, conversations, messages.	FAISS stores Dense vectors plus LangChain docstore entries containing chunk text and lookup metadata.
Link key SQLite chunk primary key is stored as chunk_id in LangChain Document metadata and as FAISS docstore ID.	vector_id SQLite also records current FAISS position for inspection and migration; positions can change after rebuilding.

Why citation is trustworthy

FAISS result se chunk_id milta hai. Backend us ID ko SQLite mein resolve karta hai. Agar row missing ya selected document se mismatch ho, result reject hota hai. Filename, page number, chunk text aur chunk index model se nahi pooche jaate; ye database se attach hote hain.

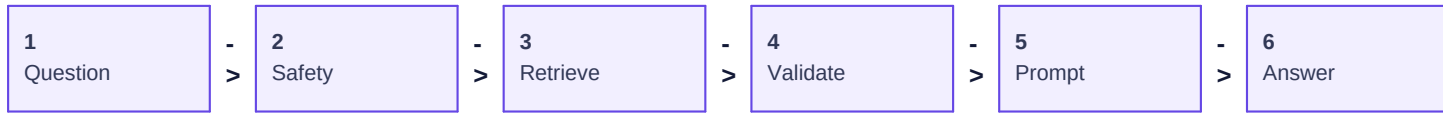
```
FAISS metadata: {
  chunk_id: 142, document_id: 7,
  document_name: 'guideline.pdf',
  page_number: 4, chunk_index: 12
}
SQLite lookup: document_chunks.id = 142
```

Deletion consistency Document delete par retained vectors ko LangChain FAISS rebuild/delete path se compact kiya jaata hai, remaining vector_id values update hote hain, DB rows cascade delete hote hain, aur PDF file remove hoti hai.

INTERVIEW NOTES

7. Question se citation-backed answer tak

Online query path latency-sensitive hai, isliye steps deterministic aur bounded rakhe gaye hain.



- ChatRequest question, selected document_ids aur optional conversation_id validate karta hai.
- Safety service high-risk keywords detect karta hai before model invocation.
- Only indexed selected IDs valid set mein jaate hain.
- LangChain FAISS query ko same OpenAIEmbeddings model se vectorize karta hai.
- similarity_search_with_score metadata filter se selected documents restrict karta hai.
- Normalized vectors ke squared L2 distance ko cosine-style score mein convert kiya jaata hai.
- Low-score matches reject hote hain; maximum top 5 chunks use hote hain.
- Retrieved chunk IDs SQLite se resolve hote hain; stale entries silently citation nahi ban sakte.
- Context Source 1, Source 2 labels ke saath ChatPromptTemplate mein format hota hai.
- ChatOpenAI temperature 0 par invoke hota hai; final answer safety notice aur citations ke saath return hota hai.
- Conversation aur user/assistant messages same DB session mein save hote hain.

No context path

Agar selected documents indexed nahi hain ya score threshold pass nahi hota, LLM call skip hoti hai aur exact fallback milta hai: I could not find enough information in the uploaded documents.

Response contract

```
{ answer, conversation_id, safety_notice, citations: [
  { document_id, document_name, page_number,
    chunk_index, chunk_text, score }
] }
```

INTERVIEW NOTES

8. Hallucination control aur medical safety

RAG hallucination ko reduce karta hai, eliminate nahi. Is distinction ko clearly bolna important hai.

Context-only prompt Model ko outside knowledge use karna aur facts invent karna explicitly forbidden hai.	Similarity threshold Weak or unrelated chunks model tak nahi pahunchte.
Fixed fallback Insufficient context mein guess karne ke bajay deterministic refusal milta hai.	Temperature zero Output variability reduce hoti hai; grounding guarantee nahi banti.
Database citations Citation metadata model-generated nahi hai; retrieved rows se attach hota hai.	Safety assessment Dosage, diagnosis, chest pain, emergency, pregnancy, child, treatment jaise intents flag hote hain.

Risky query ke liye behavior

- Prompt direct diagnosis, dosage decisions, emergency instructions aur treatment plans forbid karta hai.
- High-risk question par licensed medical professional consult karne ka deterministic warning add hota hai.
- Potential emergency symptoms par local emergency services contact karne ki warning hoti hai.
- Har answer document-only aur not-medical-advice note ke saath end hota hai.

Honest interview answer

I would never claim that RAG makes the model medically safe. It improves grounding and traceability; clinical validation, policy review, access control, monitoring, and human oversight are still required.

Strict prompt summary

- Answer only from retrieved context
- Do not use outside knowledge or invent facts
- Use exact insufficient-context fallback
- No diagnosis, dosage, emergency, or treatment decisions
- Add professional-care warning for high-risk intent
- Cite sources inline and end with medical disclaimer

INTERVIEW NOTES

9. Codebase map: kaunsa file kya karta hai

Interview mein folder structure explain karne se clean architecture aur separation of concerns demonstrate hota hai.



app/main.py App lifecycle, table initialization, CORS, routers, error handlers, legacy FAISS migration.	core/config.py Typed environment configuration through pydantic-settings; no hardcoded secrets.
db/models.py Documents, document_chunks, conversations, messages relationships and statuses.	api/routes HTTP-only layer. Dependencies leta hai, service call karta hai, response schema return karta hai.
document_service.py Ingestion transaction, status changes, rollback cleanup, deletion orchestration.	langchain_loader.py PyMuPDFLoader adapter and page metadata normalization.
langchain_chunker.py Recursive splitter configuration and global chunk indexes.	langchain_vector_store.py Embeddings, FAISS, filtering, safe persistence, rebuild, migration.
langchain_rag_service.py Retrieval, SQLite resolution, prompt, ChatOpenAI, citations, conversation saving.	safety.py Deterministic high-risk detection and mandatory safety notices.

INTERVIEW NOTES

10. API aur frontend integration

Frontend contract stable rakhna refactor ka important design constraint tha.

GET	/health	Backend liveness
GET	/api/documents	Document list and statuses
POST	/api/documents/upload	PDF ingestion
GET	/api/documents/{id}	Metadata and chunk summaries
DELETE	/api/documents/{id}	DB + file + vector deletion
POST	/api/chat	Grounded cited answer

Frontend communication

- Axios client VITE_API_BASE_URL use karta hai; backend origin CORS se allow hota hai.
- TanStack Query document list, stats aur conversation server state manage karta hai.
- Upload progress browser request progress se show hota hai; backend processing status separate hai.
- Backend snake_case response ko frontend adapter camelCase view models mein normalize karta hai.
- VITE_USE MOCKS=false real backend activate karta hai; API keys frontend env mein nahi jaati.

Refactor benefit

LangChain refactor ke baad routes aur response schemas unchanged rahe. Isse frontend mein breaking change nahi aaya - good separation of concerns ka practical proof.

INTERVIEW NOTES

11. Common interview questions - core RAG

Answers ko ratna nahi; structure samjho: definition -> project usage -> trade-off.

Q1. RAG kya hai?

RAG means Retrieval Augmented Generation. Pehle trusted knowledge base se relevant context retrieve hota hai, phir LLM us context se answer generate karta hai. Mere project mein PDFs knowledge base hain, LangChain FAISS retrieval karta hai, aur ChatOpenAI grounded answer deta hai.

Q2. Fine-tuning ke badle RAG kyun?

RAG frequently changing private documents ke liye better hai, citations de sakta hai, aur retraining ke bina new PDFs add ho sakti hain. Fine-tuning behavior/style ya task adaptation ke liye useful hai, factual document lookup ke liye primary choice nahi.

Q3. Keyword search ke badle embeddings kyun?

Keyword search lexical match dekhta hai; embeddings semantic similarity capture karte hain. User exact words use na kare tab bhi relevant concept retrieve ho sakta hai. Production mein hybrid BM25 plus vector search aur better ho sakta hai.

Q4. FAISS kya karta hai?

FAISS high-dimensional vectors mein nearest-neighbor search optimize karta hai. Yeh LLM nahi hai aur answer generate nahi karta; sirf query ke semantically closest chunks return karta hai.

Q5. Top-k aur threshold mein difference?

Top-k maximum results limit karta hai. Threshold minimum relevance quality enforce karta hai. Sirf top-k use karne par irrelevant chunks bhi aa sakte hain; threshold weak results reject karta hai.

Q6. Page-aware chunking kyun?

Citation ke liye page provenance preserve hota hai aur ek chunk multiple pages mix nahi karta. Isse source preview aur page number reliable rehte hain.

INTERVIEW NOTES

12. Common interview questions - architecture and backend

Yahan interviewer production thinking aur failure handling evaluate karega.

Q7. Route files thin kyun rakhe?

Route ka kaam HTTP input/output, dependency injection aur status codes hai. Business logic services mein hone se unit testing, reuse, provider migration aur debugging easy hoti hai.

Q8. SQLite aur FAISS dono kyun?

SQLite relational metadata, lifecycle status, chunks aur conversations ke liye suitable hai. FAISS dense vector similarity ke liye optimized hai. SQLite truth hai; FAISS derived search index hai.

Q9. DB aur vector index inconsistent ho gaye to?

Ingestion transaction rollback ke saath newly added vector documents remove karta hai. Retrieval IDs ko SQLite se resolve karta hai, so stale vectors citation nahi bante. Deletion index compact/rebuild karti hai. Larger scale par transactional outbox and background reconciliation add karunga.

Q10. Synchronous ingestion ki limitation?

Large PDF aur embedding API request upload response ko slow bana sakti hai. Next step Redis/Celery worker hoga: upload immediately processing return kare, worker retryable stages execute kare, frontend polling or WebSocket se status update le.

Q11. LangChain kyun, direct SDK kyun nahi?

LangChain loader, splitter, embeddings, vector store aur prompt interfaces standardize karta hai. Direct SDK simpler small use case mein fine hai. Maine LangChain use karke replaceability li, but domain logic explicit rakha to avoid unnecessary abstraction.

Q12. Existing API contract kaise preserve kiya?

Service container ke behind custom RAG components replace kiye. Routes, Pydantic schemas, SQLAlchemy tables aur frontend adapters untouched rahe. Tests ne response contract verify kiya.

INTERVIEW NOTES

13. Trade-offs aur scaling roadmap

Senior answer mein current decision ke saath limitation aur migration path bhi hona chahiye.

SQLite limitation Single-node writes aur concurrency limited. Multi-user production ke liye PostgreSQL.	Local FAISS limitation Process-local file, horizontal scaling aur transactional consistency difficult.
Local PDF storage Multiple app instances share nahi kar sakte. S3-compatible encrypted object storage better.	No background jobs Large ingestion request timeout risk. Celery/RQ plus Redis and idempotent tasks.
Dense-only retrieval Exact medical terminology miss ho sakti hai. Hybrid BM25 + vector search.	No reranker Top vector matches always best evidence nahi. Cross-encoder reranking improve precision.

PostgreSQL + pgvector migration

- DATABASE_URL PostgreSQL par switch karo aur Alembic migration add karo.
- document_chunks table mein vector column add karke current chunks re-embed/backfill karo.
- LangChainFAISSVectorStore ko pgvector-backed integration/repository se replace karo.
- document_id filtering same vector SQL query mein karo; HNSW index add karo after benchmarking.
- FAISS index aur JSON docstore remove karo once PostgreSQL single transactional source ban jaye.
- Routes, schemas, safety service aur most RAG orchestration same reh sakte hain.

Best scaling line

I would scale the workflow before scaling the model: background ingestion, observability, isolation, retrieval quality, and evaluation usually matter more than immediately choosing a larger LLM.

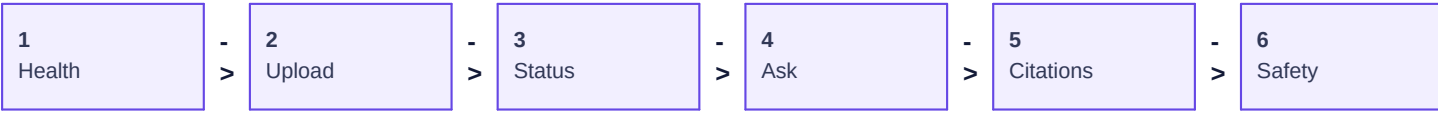
Production additions

- JWT/OAuth authentication and strict per-user document isolation
- Rate limiting, audit logs, encryption, malware scanning, retention policies
- Structured traces for load, split, embed, retrieve, prompt, and model latency
- RAGAS plus curated medical question-answer regression dataset
- Prompt/model/version tracking and provider fallback strategy

INTERVIEW NOTES

14. Live demo ka script

Interview demo ko random clicks nahi, ek clear story ki tarah present karo.



Demo sequence

- Swagger mein GET /health show karo: backend available hai.
- Known text PDF upload karo aur processing -> indexed state explain karo.
- Document detail mein chunks aur page numbers show karo.
- Ek answerable question pucho; citation card, page, preview aur score show karo.
- Ek unrelated question pucho; insufficient-context fallback show karo.
- Ek dosage/treatment question pucho; high-risk safety warning show karo.
- Document select/deselect karke prove karo retrieval selected sources tak restricted hai.

Demo preparation

Ek small deterministic PDF rakho jisme 2-3 clearly answerable facts aur page numbers hon. Provider quota, .env, backend, frontend aur index directory demo se pehle verify karo.

Agar demo fail ho

Upload failed MIME/signature, size, encrypted/scanned PDF, storage permissions, provider config check karo.	No citations Document indexed status, selected IDs, threshold, metadata JSON, embedding model consistency check karo.
Provider error API key, base URL /v1 format, model availability, timeout, quota and logs check karo.	Wrong answer Retrieved chunks inspect karo first. Retrieval bad hai ya generation bad hai - diagnosis separate karo.

FINAL REVISION

15. Last-minute revision cheat sheet

Interview se 10 minute pehle sirf yeh page revise karo.

One-line project

Medical PDFs se selected-document, citation-backed, safety-aware question answering using FastAPI and LangChain.

Ingestion

Validate -> store -> PyMuPDFLoader -> recursive chunks -> SQLite IDs -> embeddings -> FAISS.

Retrieval

Question embedding -> selected-doc filter -> scored top-k -> threshold -> SQLite validation.

Generation

Source-labelled context -> ChatPromptTemplate -> ChatOpenAI temperature 0.

Citations

Database-derived filename, page, chunk, preview, score. Never trust model-generated metadata.

Safety

Context only, fixed fallback, risk detector, no diagnosis/dosage/treatment, professional warning.

Persistence

SQLite is truth; FAISS is derived index; chunk_id links both.

Clean architecture

Routes -> schemas/dependencies -> services -> storage/DB/provider integrations.

Scale next

Postgres + pgvector, workers, auth/isolation, hybrid retrieval, reranking, evaluation.

Five lines jo confidently bolni hain

- RAG reduces hallucination by grounding generation in retrieved context, but it does not guarantee correctness.
- SQLite is my source of truth; FAISS is a rebuildable semantic index.
- Citations are attached from retrieved database metadata, not invented by the LLM.
- LangChain handles replaceable RAG components; application code owns safety, transactions, and contracts.
- The next production step is background ingestion plus PostgreSQL/pgvector and user isolation.

Final advice

Answer ko is order mein structure karo: what -> why -> how in this project -> limitation -> next improvement. Isi se junior answer production answer ban jaata hai.